Ayame: Fast Logging for Serial Safety Net

Shun Sasaki[1,a)] Takayuki Tanabe[1,b)] Hideyuki Kawashima[2,c)]

Abstract: Concurrency control is essential for database access, and the Serial Safety Net (SSN) represents one of the state-of-the-art methods in this field. Integrating SSN with logging ensures that transaction processing is recoverable. While P-WAL is an efficient logging method that leverages parallel I/O extensively, it still suffers from certain limitations. To overcome these drawbacks, this paper proposes a novel logging method, Ayame. The proposed method incorporates three optimization techniques: the elimination of the separate WAL-side global LSN allocation, dependency analysis, and asynchronous I/O. Experimental results demonstrate that Ayame achieves up to a 4.7-fold performance improvement over P-WAL in YCSB-A.

1. Introduction

1.1 Motivation

Transaction processing is a core abstraction behind data-intensive applications such as banking, payment processing, inventory management, ticket reservation, and online services that update shared state under high concurrency. A transaction processing system must provide both correct concurrent execution and crash recovery; in this sense, transaction processing consists of concurrency control (CC) and recovery [1]. A large body of work has studied scalable CC for multicore main-memory databases, including Silo, ERMIA, TicToc, SI, SSI, and SSN-style protocols [2], [3], [4], [5], [6], [7]. These protocols improve isolation and scalability by reducing centralized validation, timestamp, and version-management bottlenecks. In contrast, the recovery side of modern CC protocols has been explored less systematically: when a high-performance CC protocol is combined with crash durability, it is often unclear which recovery protocol preserves both safety and scalability.

Write-ahead logging (WAL) is the standard foundation for database recovery, with ARIES being the classical design [8], [9]. Modern systems have revisited WAL for multicore and fast-storage environments. SiloR parallelizes logging, checkpointing, and recovery for Silo; Passive Group Commit reduces synchronization overhead on non-volatile memory; and P-WAL partitions the log into multiple streams [10], [11], [12]. Although these systems differ in the target storage device, log format, and recovery procedure, they share an important lesson: writing log records in parallel is essential for making durability compatible with multicore transaction processing. At the same time, strong multiversion protocols such as SI, SSI, and SSN introduce timestamp and dependency structures that are not fully reflected in conventional parallel logging designs. The motivation of this work is therefore to clarify how P-WAL-style parallel logging should be combined with such timestamp-based CC protocols, especially SSN, without unnecessarily reintroducing global ordering and durable-prefix bottlenecks.

1.2 Problem

We target SSN because it is the most complex of the SI-family protocols we consider: it certifies serializability using dependency metadata, while the underlying engine still uses timestamped multiversion execution. As the baseline recovery protocol, we start from P-WAL, which is also used as a foundation of Shirakami, the transaction processing mechanism of Tsurugi [12], [13]. P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log streams. However, combining P-WAL with SSN exposes three problems.

P1. Multiple Global Counter Accesses: First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or maintain a global durable prefix. This duplicates the ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

P2. Waiting for Data Transfer: Second, workers can spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If worker threads wait directly for log flushes or durable-prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

[1] Graduate School of Media and Governance, Keio University, Japan
[2] Faculty of Environment and Information Studies, Keio University, Japan
[a)] shun.sasaki@keio.jp
[b)] tanab@keio.jp
[c)] river@sfc.keio.ac.jp

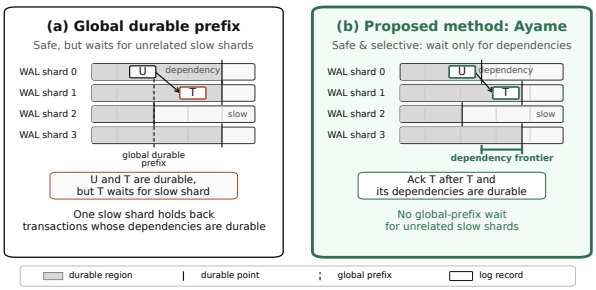

Fig. 1 Durable acknowledgment policies in parallel WAL. Global-prefix acknowledgment is safe but can delay transactions whose dependencies are already durable because it waits for unrelated slow shards. Local-only acknowledgment avoids this delay but is unsafe. Ayame uses dependency-closed acknowledgment, acknowledging a transaction only after its own log record and dependency frontier are durable.

P3. Total History: Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

Figure 1 summarizes these alternatives. Global-prefix acknowledgment is safe but may wait for unrelated slow shards. Local-only acknowledgment avoids such waits but can acknowledge a transaction before its dependencies are recoverable. The desired point is dependency-closed acknowledgment: acknowledge a transaction after its own log record and the transactions it depends on are durable.

In asynchronous durability protocols, logical commits and durable commit acknowledgments must be distinguished. A worker may continue after enqueueing a log record, but the transaction is not durable from the client's perspective until the system returns a durable acknowledgment. Therefore, this paper uses *durable-acknowledgment throughput* as the main throughput metric and reports it together with pending durable commits and tail latency.

1.3 Approach

This paper proposes *Ayame*, a timestamp-integrated, dependency-closed parallel WAL protocol for main-memory transaction processing. Ayame targets timestamp-based engines such as ERMIA, where the transaction engine already computes a logical commit order. Its goal is not merely to maximize raw logical commit throughput, but to return durable commit acknowledgments safely without unnecessary global-prefix waiting.

Ayame addresses the three problems above with three corresponding techniques.

A1. Commit timestamps as logical LSNs. Ayame reuses the SSN/ERMIA commit timestamp as the logical LSN for recovery. WAL shards still maintain local physical positions, but the WAL layer does not allocate an additional global logical order. This removes duplicated global ordering between CC and recovery and eliminates WAL-side separate global LSN allocation on the commit path.

A2. Worker/flusher/committer separation. Ayame decouples transaction execution from durable acknowledgment. Workers execute and validate transactions, assign commit timestamps, and enqueue log records. Flushers batch and persist shard-local logs. Committers observe durable-frontier advances and return durable acknowledgments. This pipeline keeps workers from directly blocking on `fdatasync` or on durable-prefix notification waits.

A3. Dependency-closed durable acknowledgment. Ayame replaces global-prefix acknowledgment with dependency-closed acknowledgment. For each transaction, Ayame maintains a dependency frontier that summarizes the log positions that must be recoverable before the transaction can be acknowledged. A transaction is acknowledged only when its own log record and this dependency frontier are durable. This preserves recovery safety while avoiding waits for unrelated WAL shards.

We implement Ayame in an ERMIA-style engine and evaluate it against P-WAL using SSN and YCSB workloads. The detailed experimental methodology, figures, and tables are presented in Section 4.

1.4 Organization

The rest of this paper is organized as follows. Section 2 describes transaction processing, including serial-safety-net and parallel logging protocols. Section 3 presents Ayame, which is a fast logging protocol. Section 4 describes the evaluation of Ayame compared with the P-WAL protocol using SSN. Section 5 discusses related work. Section 6 finally concludes this paper.

2. Preliminaries

2.1 Concurrency Control
2.2 Timestamp-based Protocols
2.2.1 Serial Safety Net
2.3 Write Ahead Logging for Recovery
2.4 Logging
2.4.1 P-WAL

P-WAL is attractive because it removes the single-WAL insertion bottleneck by assigning transactions to multiple log streams. However, combining P-WAL with SSN exposes three problems.

P1. Multiple Global Counter Accesses: First, P-WAL can introduce frequent global-counter accesses. Even if the CC layer already assigns a commit timestamp for SSN validation and serialization, the recovery layer may allocate a separate global LSN or maintain a global durable prefix. This duplicates the ordering work already performed by the CC layer and adds shared-memory synchronization to the commit path.

P2. Waiting for Data Transfer: Second, workers can

Algorithm 1 Per-Thread WAL Commit (P-WAL)

```
1: Shared: global LSN counter L; durable prefix P, the largest
   ℓ such that every record with LSN ≤ ℓ is durable
2: function PwalCommit(T, worker w)
3:     for each write e ∈ T.write_set do
4:         append ⟨FetchInc(L), e⟩ to stream[w] ▷ global counter
5:     end for
6:     T.ℓ ← FetchInc(L)            ▷ commit LSN, global counter
7:     append commit record ⟨T.ℓ⟩ to stream[w]
8:     Write(stream[w]); Fdatasync(stream[w]) ▷ worker blocks
   on storage
9:     AdvanceDurablePrefix                  ▷ recompute P
10:    while P < T.ℓ do              ▷ global-prefix acknowledgment
11:        AdvanceDurablePrefix
12:    end while
13:    Acknowledge(T)
14: end function
```

spend a long time waiting around `fdatasync`. P-WAL parallelizes log streams, but a transaction cannot be acknowledged until the corresponding durable condition is satisfied. If worker threads wait directly for log flushes or durable-prefix updates, storage synchronization and notification latency reduce the effective scalability of the CC protocol.

P3. Total History: Third, global-prefix acknowledgment waits for log records that are unrelated to the transaction being acknowledged. This rule is safe, but overly conservative: if one WAL shard is slow, transactions on other shards may be delayed even when they have no dependency on transactions logged by the slow shard. The opposite design point, local-only acknowledgment, avoids this delay but is unsafe. If transaction T reads a value written by transaction U, acknowledging T before U is recoverable may allow a crash recovery state that contains T without the state on which T depends.

3. Proposal: Ayame

Ayame is a logging method that integrates durable write-ahead logging into SSN while avoiding the three problems of P-WAL identified in Section 2. It combines three techniques. First, it reuses the commit timestamp that SSN already assigns as the logical log sequence number, eliminating the separate WAL-side global counter (addresses **P1**). Second, it splits commit processing into worker, flusher, and committer roles so that worker threads do not block on `fdatasync` or durable-ack waiting, subject only to a bounded backpressure limit (addresses **P2**). Third, it acknowledges a transaction using a *dependency-closed durable frontier* instead of a global durable prefix, so that a transaction waits only for the log records it actually depends on (addresses **P3**).

We use the following notation. A frontier F is a vector of per-shard log positions. $F \sqcup F'$ denotes the element-wise maximum, $F \leq F'$ holds iff $F[i] \leq F'[i]$ for every shard i, and $\perp$ denotes the empty frontier whose positions are all zero. D denotes the durable vector, where $D[i]$ is the largest log position of shard i that is known to be durable; each $D[i]$ advances monotonically.

3.1 Commit Timestamp as Logical LSN

In SSN, a committing transaction T is assigned a commit timestamp $T.c$ that fixes its position in the serialization order. A separate WAL-side global LSN, as used by P-WAL, would redundantly re-encode the same logical order and add a second global-counter access to every commit. Ayame instead reuses $T.c$ directly as the logical LSN that orders recovery replay. No WAL-side global sequence number is allocated; in our evaluation the per-transaction WAL-side global-counter access drops to zero.

Removing the global *logical* LSN does not remove physical log addressing. Each WAL shard keeps a local position (a per-shard sequence number and byte offset) that locates a record within its log file. What Ayame eliminates is the duplicated global ordering counter, not the per-shard physical position.

3.2 Worker, Flusher, and Committer Separation

Ayame divides commit processing into three roles. A *worker* executes the transaction, validates it with SSN, assigns the commit timestamp, appends the log record to its own WAL shard, publishes durability metadata, and registers a durable-acknowledgment request. The worker then returns to its next transaction; it does not call `fdatasync` and does not wait for a durable acknowledgment. A *flusher* owns one WAL shard, batches the queued records, writes them, calls `fdatasync`, and advances the shard's durable position. A *committer* returns durable acknowledgments to clients once the durable condition of a registered transaction is met. Workers therefore do not block on storage synchronization or durable-ack waiting, and keep executing transactions at full concurrency even though every update commit must still be made durable; the cost of making the log durable is paid by the flusher and committer. The one exception is a bounded backpressure limit: to cap memory use and acknowledgment latency, a worker stalls only if the number of registered but not-yet-acknowledged durable commits reaches a configured maximum. Accordingly, we report throughput as the number of durable acknowledgments returned to clients, not the number of logical commits.

Algorithm 2 shows the worker-side commit protocol. The commit timestamp assigned for SSN validation doubles as the logical LSN (line 2). After validation, the worker enqueues the record to its shard without flushing or waiting (line 16). It then forms the closed frontier F by extending its dependency frontier $T.dep$ with its own log position (line 17), publishes F to the versions it wrote and read (lines 19 and 22), and registers F for durable acknowledgment (line 24) before returning a logical commit.

The flusher does not require an algorithm of its own. Each flusher owns one WAL shard, repeatedly collects the records queued by workers, writes them as a single batch, calls `fdatasync` once for the batch, and then advances the shard's durable position by invoking OnDurableAdvance (Algorithm 4). Batching many records per `fdatasync` is what raises the number of commits made durable per syn-

Algorithm 2 Worker-Side Commit Protocol (Ayame)

```
 1: function COMMIT(T)
 2:     T.c ← commit timestamp assigned by SSN   ▷ logical LSN
 3:     if SSNVALIDATE(T) fails then
 4:         abort T; return
 5:     end if
 6:     if T's write set is empty then                ▷ read-only
 7:         if frontier collection is skipped or T.dep = ⊥ then
 8:             ACK(T)                            ▷ read-only fast path
 9:         else
10:             REGISTER(T.dep, T)                    ▷ Algorithm 4
11:         end if
12:         return logical read-only commit
13:     end if
14:     s ← WAL shard assigned to this worker
15:     r ← BUILDRECORD(T.c, T.dep, T.write_set)
16:     T.seq ← ENQUEUE(wal[s], r)   ▷ append only; no flush, no wait
17:     F ← T.dep;   F[s] ← max(F[s], T.seq)   ▷ closed frontier
18:     for all versions v created by T do
19:         v.wf ← F
20:     end for
21:     for all versions v read by T do
22:         v.rf ← v.rf ⊔ F                       ▷ monotone merge
23:     end for
24:     REGISTER(F, T)   ▷ durable ack deferred to the committer
25:     return logical commit
26: end function
```

chronization and reduces flush pressure relative to P-WAL's per-transaction synchronization.

3.3 Dependency-Closed Durable Acknowledgment

The remaining question is when a logically committed transaction may be acknowledged as durable. Ayame answers this with a frontier attached to each version. A version carries a *write frontier* wf, the closed frontier of the transaction that created it, and a *read frontier* rf, which conservatively accumulates the closed frontiers of read-write transactions that read it. During execution, a transaction accumulates these frontiers into its dependency frontier $T.dep$, as shown in Algorithm 3: reading a version makes T depend on the writer of that version (line 9), and overwriting a version makes T additionally depend on the readers of the overwritten version (line 13). Transactions known to be read-only skip collection entirely and take the snapshot-isolation fast path.

At commit, the closed frontier F (line 17 of Algorithm 2) extends $T.dep$ with T's own log position. By induction over the dependency order, F therefore contains the log positions of the transitive closure of the read-from and overwrite dependencies of T. Publishing F to the versions T wrote and read lets future transactions inherit it through Algorithm 3.

Algorithm 4 shows the committer side. REGISTER parks T on each shard whose durable position has not yet reached the requirement of F (line 5); if no shard is behind, T is acknowledged immediately (line 8). ONDURABLEADVANCE, invoked by a flusher after `fdatasync`, advances the durable position of one shard and wakes the parked transactions whose requirement on that shard is now satisfied, releasing the acknowledgment once a transaction's last outstanding shard becomes durable (line 17). REGISTER and ONDURABLEADVANCE execute under a single lock so that a durable advance cannot slip between the durability test and the parking step.

The acknowledgment rule is exactly $F \le D$: a transaction is acknowledged once its own record and every record it transitively depends on are durable. We now make the safety of this rule precise. For a committed update transaction U, let s_U and $U.seq$ be the shard and position of its log record,

Algorithm 3 Dependency Collection (Ayame)

```
 1: function BEGIN(T)
 2:     T.dep ← ⊥
 3: end function

 4: function READVERSION(T, v)
 5:     add v to T's read set
 6:     if T is known read-only then
 7:         return                          ▷ SI read-only fast path
 8:     end if
 9:     T.dep ← T.dep ⊔ v.wf                       ▷ the writer of v
10: end function

11: function OVERWRITEVERSION(T, v)
12:     add the new version to T's write set
13:     T.dep ← T.dep ⊔ v.wf ⊔ v.rf   ▷ the writer of v and its readers
14: end function
```

Algorithm 4 Durable Acknowledgment (Ayame)

```
    State: durable vector D; per-shard waitlists W[i] ordered by required position
 1: function REGISTER(F, T)                          ▷ atomic w.r.t. ONDURABLEADVANCE
 2:     T.remaining ← 0
 3:     for all shards i such that F[i] > D[i] do
 4:         T.remaining ← T.remaining + 1
 5:         insert (F[i], T) into W[i]
 6:     end for
 7:     if T.remaining = 0 then
 8:         ACK(T)      ▷ own record and all dependencies durable
 9:     end if
10: end function

11: function ONDURABLEADVANCE(i, d)      ▷ called by flusher i after fdatasync
12:     D[i] ← d
13:     while W[i] is not empty and MINKEY(W[i]) ≤ d do
14:         (need, T) ← POPMIN(W[i])
15:         T.remaining ← T.remaining − 1
16:         if T.remaining = 0 then
17:             ACK(T)
18:         end if
19:     end while
20: end function
```

so its published closed frontier F_U satisfies $F_U[s_U] \geq U.seq$. Let $\mathrm{cl}(T)$ be the set of update transactions reachable from T through read-from and overwrite dependencies, including T itself.

Lemma 1 (Dependency-closed durability) If $\textsc{Ack}(T)$ is invoked, then for every update transaction $U \in \mathrm{cl}(T)$ the WAL record of U is durable.

Proof sketch. Dependency collection (Algorithm 3) joins each accessed version's published frontier into $T.dep$, and the commit protocol sets $F = T.dep$ with $F[s_T] \geq T.seq$ (line 17). Because every published $v.wf$ and $v.rf$ is itself a closed frontier, an induction over the dependency order gives $F \geq F_U$, hence $F[s_U] \geq F_U[s_U] \geq U.seq$, for every $U \in \mathrm{cl}(T)$. $\textsc{Ack}(T)$ is invoked only when $F \leq D$ (Algorithm 4), so $D[s_U] \geq U.seq$. Since $D[i]$ is a durable prefix of shard i, the record of U is durable.

Theorem 1 (Recoverable acknowledgment) Suppose a crash occurs and recovery replays exactly the durable WAL records in logical-LSN order. If T was acknowledged before the crash, recovery never yields a state that contains the effects of T but omits the effects of some $U \in \mathrm{cl}(T)$.

Proof sketch. By Lemma 1, every $U \in \mathrm{cl}(T)$ has a durable record and is therefore replayed. Each dependency edge $U \to T$ is a serialization edge of SSN, so U's commit timestamp precedes T's; because logical LSNs equal commit timestamps, replay in logical-LSN order installs every such U before T. Hence whenever T's effects are installed, the effects of all $U \in \mathrm{cl}(T)$ are already present.

The rule is also less conservative than a global durable prefix: a transaction waits only on the shards that appear in its frontier, so an unrelated slow shard never delays it.

4. Evaluation

4.1 Implementation

We implement all three WAL systems on top of CCBench [7], a concurrency-control benchmarking framework with minimal functionality for protocol analysis. The underlying engine is an ERMIA-style multi-version store with snapshot isolation and SSN validation, using Masstree as the index and `mimalloc` as the allocator. The code is written in C++20 and compiled with GCC at the `-O3` optimization level. Our implementation is publicly available.[*1]

A key methodological point is that Single WAL, P-WAL, and Ayame are implemented in the *same* binary and selected at run time by environment flags (`CCBENCH_WAL_MODE` and `CCBENCH_WAL_DURABLE_MODE`). The concurrency-control engine, the index, the workload generator, and the pre-commit path are therefore identical across the three systems; only the WAL durability path differs. Single WAL appends to one shared log stream under a global latch and synchronizes it per commit. P-WAL gives each worker a private stream and synchronizes it per commit, acknowledging through a global durable prefix. Ayame uses the worker/flusher/committer pipeline of Section 3 with dependency-closed acknowledgment. Each WAL record carries the key, value, storage identifier, and operation type of a write, terminated by a commit record. Read-only WAL skipping is enabled for all three systems; once a transaction is known to be read-only it produces no WAL record, and Ayame additionally takes a read-only snapshot-isolation fast path that skips durability-frontier collection.

[*1] The source code is available in our public repository for reproduction.

Table 1 Default parameter settings.

Type	Parameter	Setting
DB	#records	100,000
	value size	4 B
	key distribution	uniform
TX	#operations	10
	read ratio (YCSB-A/B/C)	50/95/100 %
Run	worker threads	1,2,4,8,16,32
	measurement time	5 s
	repeats	5
Ayame	group size	16
	flush interval	50 μs
	max pending acks	65,536

4.2 Experimental Configuration

4.2.1 Environment

Experiments were conducted on a server with two sockets of Intel(R) Xeon(R) Gold 5418N CPUs. Each socket had 24 physical cores with two hardware threads per core, for 48 physical and 96 logical cores in total, organized as two NUMA nodes. Each core had a 48 KiB private L1d cache and a 2 MiB private L2 cache, and each socket shared a 45 MiB L3 cache. The DRAM capacity was 256 GB. The machine ran Ubuntu 22.04.5 LTS (Linux kernel 5.15), and WAL files were placed on a locally attached SSD formatted with ext4. Worker threads were pinned to distinct logical cores with `sched_setaffinity`.

4.2.2 Benchmark and Workloads

All experiments used the Yahoo! Cloud Serving Benchmark (YCSB) [14]. We evaluated three representative workloads: YCSB-A (50% read, 50% update), YCSB-B (95% read, 5% update), and YCSB-C (read-only). These workloads exercise the WAL along a spectrum from update-heavy to read-only and are widely used to evaluate concurrency-control protocols. Each transaction issued a fixed number of ten operations over a database of 100,000 records with 4-byte values, and keys were drawn from a uniform distribution.

4.2.3 Systems Compared and Threads

The x-axis of every throughput figure is the number of transaction *worker* threads. Single WAL and P-WAL use only worker threads. Ayame additionally runs background flusher and committer threads, so we also report its total active thread count. Table 2 lists the worker-to-flusher/committer mapping used for Ayame; for example, at 32 worker threads Ayame uses 7 flusher threads and one committer thread, for 40 active threads.

4.2.4 Metric

The primary metric is *durable-acknowledgment through-*

Table 2 Ayame background-thread mapping.

Worker	Flusher	Committer	Total active
1	1	1	3
2	1	1	4
4	1	1	6
8	2	1	11
16	4	1	21
32	7	1	40

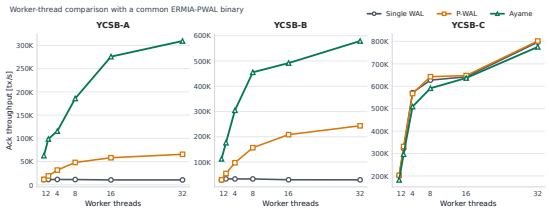

Fig. 2 Durable-acknowledgment throughput by transaction worker threads. Ayame improves update and mixed read-write workloads by combining parallel WAL logging, dependency-closed durable acknowledgment, and timestamp-integrated logical ordering.

put: the number of transactions acknowledged as durable to the client, divided by the measurement time. All systems are measured at the same stop barrier, and acknowledgments drained during shutdown are excluded. For the worker-wait systems (Single WAL and P-WAL) a logical commit and a durable acknowledgment coincide, so the metric has the same meaning across the three systems. We additionally report the p99 durable acknowledgment latency and the number of pending (not-yet-acknowledged) durable commits.

4.3 Results

We evaluate Ayame using the worker-thread sweep of Table 1. At 32 worker threads on YCSB-A, Single WAL achieves 10K durable acknowledgments per second and P-WAL achieves 66K, whereas Ayame achieves 309K durable acknowledgments per second. On YCSB-B, Single WAL achieves 29K and P-WAL achieves 243K, whereas Ayame achieves 579K durable acknowledgments per second. These results correspond to 4.7× and 2.4× improvements over P-WAL on YCSB-A and YCSB-B, respectively. Ayame also keeps the number of pending durable commits small at 32 worker threads: 145 pending commits on YCSB-A and 66 pending commits on YCSB-B.

Figure 2 shows the worker-thread comparison. The results show that Ayame improves update and mixed read-write workloads, where durable logging is still required on every update commit. We also evaluate YCSB-C, where all transactions are read-only and WAL records are skipped. In this case, durability work largely disappears; after using the same binary and a read-only empty-frontier fast path, Single WAL, P-WAL, and Ayame perform similarly at 32 worker threads. We therefore use YCSB-C as a sanity check for read-only bookkeeping overhead rather than as evidence of WAL persistence performance.

Ayame's timestamp integration is primarily a design simplification: it removes duplicated logical ordering between concurrency control and WAL durability by reusing ERMIA's commit timestamp as the logical recovery order. We do not rely on timestamp integration alone as a direct throughput optimization; the main performance effect comes from parallel logging, dependency-closed durable acknowledgment, and separating workers from durable-wait processing.

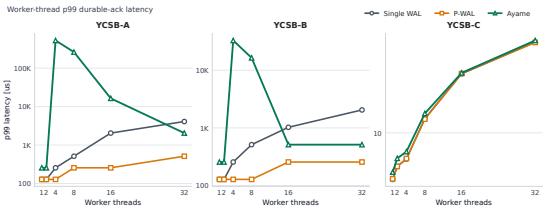

Fig. 3 P99 durable-acknowledgment latency by transaction worker threads.

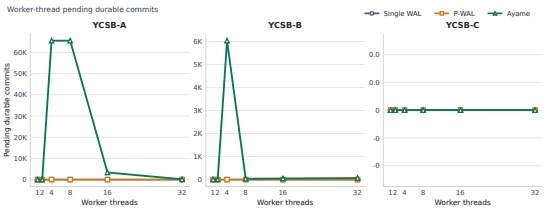

Fig. 4 Pending durable commits by transaction worker threads.

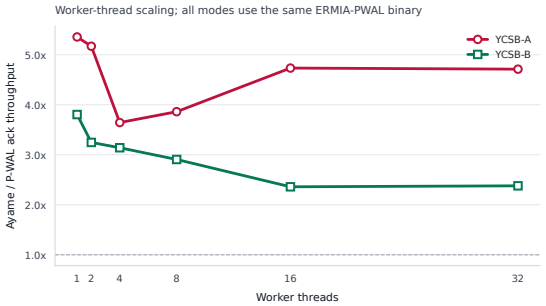

Fig. 5 Ayame throughput relative to P-WAL by transaction worker threads.

Table 3 End-to-end YCSB results at 32 transaction worker threads.

Workload	System	Ack tps	p99 μs	Pending	fdatasync	Tx
YCSB-A	Single WAL	10.5K	4096	0	42135	
YCSB-A	P-WAL	65.7K	512	0	262491	
YCSB-A	Ayame	309.5K	2048	145	88018	
YCSB-B	Single WAL	29.4K	2048	0	47239	
YCSB-B	P-WAL	243.4K	256	0	390814	
YCSB-B	Ayame	579.1K	512	66	125299	
YCSB-C	Single WAL	796.4K	40	0	0	
YCSB-C	P-WAL	801.7K	40	0	0	
YCSB-C	Ayame	775.4K	41	0	0	

Ack tps is durable-acknowledgment throughput. Ayame uses 7 flusher threads and 1 committer thread at 32 workers.

Table 4 Ayame throughput relative to P-WAL at 32 transaction worker threads.

Workload	P-WAL tps	Ayame tps	Speedup	Ayame p99 μs	Ayan
YCSB-A	65.7K	309.5K	4.71×	2048	
YCSB-B	243.4K	579.1K	2.38×	512	
YCSB-C	801.7K	775.4K	0.97×	41	

Table 3 summarizes the main 32-worker end-to-end results used in the figures. Table 4 extracts the Ayame-vs-P-WAL speedups from the same measurements.

Table 5 summarizes the 32-worker perf-stat analysis. Ta-

Table 5 Perf-stat summary at 32 transaction worker threads.

Workload	System	Ack tps	CPU cores	Cycles/tx	Instr/tx	IPC	Ctx sw.	Tx/sync
YCSB-A	Single WAL	9.8K	1.2	308526	674082	2.18	128118	1.0
YCSB-A	P-WAL	68.5K	19.0	698368	415055	0.59	1537033	1.0
YCSB-A	Ayame	310.6K	26.7	213987	139150	0.65	3203414	14.7
YCSB-B	Single WAL	27.2K	1.1	98314	165785	1.69	136497	2.5
YCSB-B	P-WAL	249.5K	18.0	182056	103894			
YCSB-B	Ayame	571.1K	31.2	137447	63383			
YCSB-C	Single WAL	796.0K	40.1	130591	28272			
YCSB-C	P-WAL	793.1K	40.1	131230	28278			
YCSB-C	Ayame	810.1K	40.1	128448	32517			

Table 6 YCSB-B perf-stat worker scaling.

Workers	Single tps	P-WAL tps	Ayame tps	Single CPU	P-WA		
1	27.9K	28.8K	121.3K	0.5			
2	32.3K	53.7K	198.5K	0.6			
4	32.1K	94.2K	324.5K	0.6			
8	32.0K	160.4K	434.9K	0.6		3.8	10.5
16	28.9K	208.9K	505.0K	0.7		8.7	19.7
32	29.1K	245.5K	573.0K	1.0		18.0	32.1

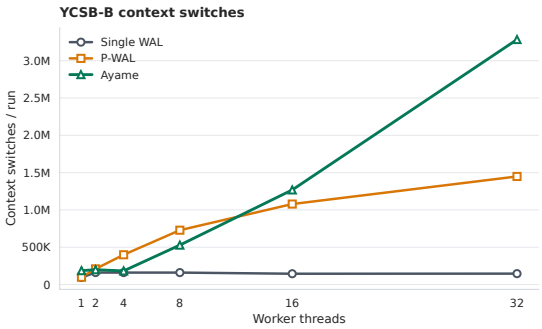

Fig. 8 Context-switch counts in the YCSB-B perf-stat worker-scaling runs.

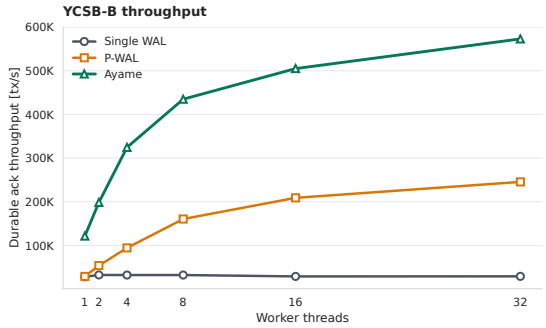

Fig. 6 YCSB-B durable-acknowledgment throughput in the perf-stat worker-scaling runs.

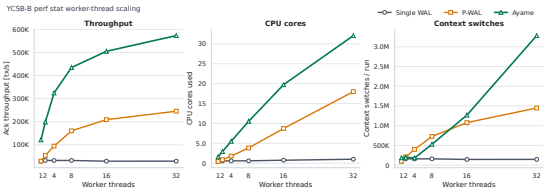

Fig. 9 Perf-stat summary for YCSB-B worker scaling.

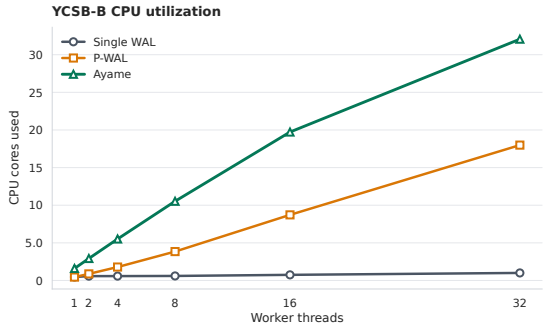

Fig. 7 CPU-core utilization in the YCSB-B perf-stat worker-scaling runs.

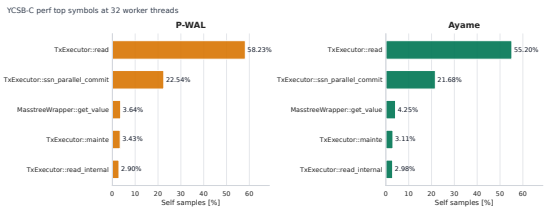

Fig. 10 Top self-time symbols for YCSB-C at 32 transaction worker threads.

ble 6 shows the worker-scaling perf-stat data for YCSB-B, where Ayame's throughput improvement is most visible.

Finally, Table 7 reports the top self-time symbols for YCSB-C. Since YCSB-C is read-only, this table is used as a sanity check that the remaining cost is read-path bookkeeping rather than WAL persistence.

5. Related Work

6. Conclusion

Acknowledgments

Table 7 Top self-time symbols for YCSB-C at 32 transaction worker threads.

System	Rank	Self %	Symbol
Single WAL	1	57.83	TxExecutor::read
Single WAL	2	22.89	TxExecutor::ssn_parallel_commit
Single WAL	3	4.09	MasstreeWrapper¡Tuple¿::get_value
Single WAL	4	3.25	TxExecutor::mainte
Single WAL	5	3.01	TxExecutor::read_internal
P-WAL	1	58.23	TxExecutor::read
P-WAL	2	22.54	TxExecutor::ssn_parallel_commit
P-WAL	3	3.64	MasstreeWrapper¡Tuple¿::get_value
P-WAL	4	3.43	TxExecutor::mainte
P-WAL	5	2.90	TxExecutor::read_internal
Ayame	1	55.20	TxExecutor::read
Ayame	2	21.68	TxExecutor::ssn_parallel_commit
Ayame	3	4.25	MasstreeWrapper¡Tuple¿::get_value
Ayame	4	3.11	TxExecutor::mainte
Ayame	5	2.98	TxExecutor::read_internal

YCSB-C is read-only; WAL bytes and fdatasync counts are zero for all systems.

References

[1] Bernstein, P. A., Hadzilacos, V. and Goodman, N.: *Concurrency Control and Recovery in Database Systems*, Addison-Wesley (1987).

[2] Tu, S., Zheng, W., Kohler, E., Liskov, B. and Madden, S.: Speedy transactions in multicore in-memory databases, *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, New York, NY, USA, Association for Computing Machinery, p. 18–32 (online), DOI: 10.1145/2517349.2522713 (2013).

[3] Kim, K., Wang, T., Johnson, R. and Pandis, I.: ERMIA: Fast Memory-Optimized Database System for Heterogeneous Workloads, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1675–1687 (online), DOI: 10.1145/2882903.2882905 (2016).

[4] Yu, X., Pavlo, A., Sanchez, D. and Devadas, S.: TicToc: Time Traveling Optimistic Concurrency Control, *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD '16, New York, NY, USA, Association for Computing Machinery, p. 1629–1642 (online), DOI: 10.1145/2882903.2882935 (2016).

[5] Ports, D. R. K. and Grittner, K.: Serializable Snapshot Isolation in PostgreSQL, *Proc. VLDB Endow.*, Vol. 5, No. 12, pp. 1850–1861 (online), DOI: 10.14778/2367502.2367523 (2012).

[6] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Efficiently Making (Almost) Any Concurrency Control Mechanism Serializable, *The VLDB Journal*, Vol. 26, No. 4, pp. 537–562 (online), DOI: 10.1007/s00778-017-0463-8 (2017).

[7] Tanabe, T., Hoshino, T., Kawashima, H. and Tatebe, O.: An analysis of concurrency control protocols for in-memory databases with CCBench, *Proc. VLDB Endow.*, Vol. 13, No. 13, p. 3531–3544 (online), DOI: 10.14778/3424573.3424575 (2020).

[8] Haerder, T. and Reuter, A.: Principles of transaction-oriented database recovery, *ACM Comput. Surv.*, Vol. 15, No. 4, p. 287–317 (online), DOI: 10.1145/289.291 (1983).

[9] Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H. and Schwarz, P.: ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging, *ACM Trans. Database Syst.*, Vol. 17, No. 1, p. 94–162 (online), DOI: 10.1145/128765.128770 (1992).

[10] Zheng, W., Tu, S., Kohler, E. and Liskov, B.: Fast Databases with Fast Durability and Recovery Through Multicore Parallelism, *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation*, OSDI '14, USENIX Association, pp. 465–477 (online), available from ⟨https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_wenting⟩ (2014).

[11] Wang, T., Johnson, R., Fekete, A. and Pandis, I.: Scalable Logging Through Emerging Non-Volatile Memory, *Proc. VLDB Endow.*, Vol. 7, No. 10, pp. 865–876 (online), available from ⟨https://www.vldb.org/pvldb/vol7/p865-wang.pdf⟩ (2014).

[12] 神谷, 孝., 川島, 英., 星野, 喬., 建部, 修., Komei, K., Hideyuki, K., Takashi, H. and Osamu, T.: P-WAL: Paralell Write-ahead Logging, **情報処理学会論文誌データベース（*TOD*）**, Vol. 10, No. 1, pp. 24–39 (online), available from ⟨https://cir.nii.ac.jp/crid/1050282812885062144⟩ (2017).

[13] Kawashima, H.: Next-generation Database Tsurugi: Concurrency Control and Recovery Systems, *IPSJ Magazine*, Vol. 66, No. 4, pp. e9–e15 (online), DOI: 10.20729/0002001646 (2025).

[14] Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R. and Sears, R.: Benchmarking cloud serving systems with YCSB, *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, New York, NY, USA, Association for Computing Machinery, p. 143–154 (online), DOI: 10.1145/1807128.1807152 (2010).